

QUIC Server-Side Migration: Full Combination Test Report

QUIC Server-Side Migration: Full Combination Test Report

Test Environment

Machine	Role	IP	Hardware
optiplex7010	Client	141.217.168.127	Desktop PC
opti7040	Primary Server	141.217.168.152	Desktop PC
homeserver2	Preferred Server	141.217.168.143	Desktop PC
redis-server	Redis (Proxmox VM)	141.217.168.200	LXC Container

All machines on the same LAN (141.217.168.0/24).

Two Research Dimensions

WHEN to Transfer (Timing Strategy)

Timing	Env Variable	Description	Trade-off
Immediate	TRANSFER_TIMING=immediate	Transfer state right after TLS handshake completes. Preferred server blocks waiting for state before listening for QUIC packets.	Simpler but always transfers state even if client never migrates. Requires state to arrive before first PATH_CHALLENGE.
Lazy	TRANSFER_TIMING=lazy	Preferred server starts listening for QUIC packets immediately. State receiver runs in a background thread. Crypto import happens on-demand when first unknown CID packet arrives.	More efficient (only imports when needed) but requires state to persist until fetched.

HOW to Transfer (Mechanism)

Mechanism	Env Variable	Pattern	Description
TCP Push	STATE_TRANSFER=tcp	Direct push	Primary opens TCP socket to preferred, sends state bytes directly. One-way, one-time, connection closed after. Lowest latency. No dependencies.
HTTP Pull	STATE_TRANSFER=http	On-demand pull	Primary starts HTTP server exposing /state endpoint. Preferred sends HTTP GET to pull state. Secrets stay in primary memory until pulled. Best security.
Redis KV	STATE_TRANSFER=redis_kv	Shared storage	Primary writes state to Redis with SET (CID-based key, configurable TTL). Preferred reads with GET, then DEL. Supports N:M server routing. Best scalability.
Redis Pub/Sub	STATE_TRANSFER=redis_ps	Event-driven push	Primary publishes state to Redis channel. Preferred subscribes and receives message. Preferred must subscribe before primary publishes. No persistence.

Results: 4 x 2 = 8 Combinations

With neqo-client (automated test)

Mechanism	Timing	Result	Send Latency	PATH_CHALLENGE	PATH_RESPONSE	Notes
TCP Push	Immediate	FAIL	612-732 us	0	0	Client closes before PATH_CHALLENGE sent
TCP Push	Lazy	PASS	464-746 us	1	1	Fastest latency
HTTP Pull	Immediate	FAIL	21-92 ms	0	0	Client closes before PATH_CHALLENGE sent
HTTP Pull	Lazy	PASS	1-27 ms	1	1	Higher latency due to HTTP overhead
Redis KV	Immediate	FAIL	816-982 us	0	0	Client closes before PATH_CHALLENGE sent
Redis KV	Lazy	PASS	722-830 us	1	1	Good balance of latency and features
Redis Pub/Sub	Immediate	FAIL	1.0-1.2 ms	0	0	Client closes before PATH_CHALLENGE sent
Redis Pub/Sub	Lazy	PASS	855-913 us	1	1	Same Redis dep as KV, worse persistence

Why Immediate fails with neqo-client: The neqo-client tool connects, sends HTTP request, and closes within ~130ms. In immediate mode, the preferred server blocks waiting for state transfer to complete before listening for QUIC packets. By the time it starts listening, the client has already sent and given up on the PATH_CHALLENGE.

With Firefox Browser (manual test, confirmed working)

Mechanism	Timing	Result	PATH_CHALLENGE	PATH_RESPONSE	Notes
TCP Push	Immediate	PASS	5+	5+	Firefox retries PATH_CHALLENGE for ~200ms
TCP Push	Lazy	PASS	5+	5+	Works because state arrives before timeout
HTTP Pull	Immediate	PASS	5+	5+	Confirmed
HTTP Pull	Lazy	PASS	5+	5+	Confirmed
Redis KV	Immediate	PASS	5+	5+	Confirmed with packet capture
Redis KV	Lazy	PASS	5+	5+	Confirmed
Redis Pub/Sub	Immediate	PASS	5+	5+	Confirmed
Redis Pub/Sub	Lazy	PASS	5+	5+	Confirmed

All 8 combinations PASS with Firefox because Firefox keeps the connection alive and retries PATH_CHALLENGE multiple times over ~200ms, giving the preferred server time to receive and import the state.

Multi-Metric Evaluation (1-5 scale, 5 is best)

Combination	Latency	Security	Scalability	Reliability	Coupling	Simplicity	Dependencies	Persistence	TOTAL
Imm + TCP Push	5	1	1	2	1	5	5	1	21
Imm + HTTP Pull	2	3	3	3	3	3	5	2	24
Imm + Redis KV	4	3	5	5	5	3	2	5	32
Imm + Redis PS	4	1	4	2	5	3	2	1	22
Lazy + TCP Push	5	2	1	3	1	3	5	1	21
Lazy + HTTP Pull	2	5	3	3	3	3	5	3	27
Lazy + Redis KV	4	3	5	5	5	3	2	5	32
Lazy + Redis PS	4	1	4	3	5	3	2	1	23

Metric Definitions

Metric	What it measures
Latency	How fast state transfers from primary to preferred. TCP Push is fastest (~700us). HTTP is slowest (~25ms).
Security	How well TLS secrets are protected. HTTP Pull is best (secrets stay in primary memory until pulled). Redis broadcasts to any subscriber (worst).
Scalability	Can it handle N primary servers and M preferred servers? Redis KV scales best with CID-based keys. TCP Push is 1:1 only.
Reliability	Does it work if a server crashes or restarts? Redis KV persists state with TTL. TCP Push loses state if connection drops.
Coupling	How independent are the servers? Redis = loosely coupled (servers don't need to know each other). TCP = tightly coupled.
Simplicity	Implementation complexity. TCP Push is simplest (raw socket). HTTP/Redis require protocol handling.
Dependencies	External infrastructure needed. TCP/HTTP need nothing. Redis requires a Redis server.
Persistence	Does state survive server restart? Redis KV with TTL = yes. TCP/Pub/Sub = no.

Recommendations

Use Case	Best Choice	Score	Why
Production	Redis KV (either timing)	32/40	Highest scalability, reliability, persistence. Supports N:M routing.
Security-critical	Lazy + HTTP Pull	27/40	Secrets stay in primary memory. No third-party storage.
Demos/testing	Immediate + TCP Push	21/40	Fastest. Simplest. No dependencies.

Use Case	Best Choice	Score	Why
Not recommended	Redis Pub/Sub	22-23	Same Redis dependency as KV but worse security and persistence.

Packet Capture Evidence

Captured on client machine using `tshark` :

```
Phase 1 – QUIC Handshake (Client ↔ Primary .152):
Packets #1-#12: TLS 1.3 handshake + HTTP/3 response
DCID to primary: 07097f4a704f62c6
```

```
Phase 2 – Migration (Client ↔ Preferred .143):
#13 Client → Preferred 1260 bytes PATH CHALLENGE (encrypted)
#18 Preferred → Client 1208 bytes PATH RESPONSE DCID=1cbe79
#19 Client → Preferred 1260 bytes PATH CHALLENGE
#21 Preferred → Client 1208 bytes PATH_RESPONSE DCID=1cbe79
... (5 challenge/response pairs total)
```

Key evidence:

- Preferred server responds with DCID=1cbe79 (same client CID used by primary)
- This proves preferred server has the same TLS keys
- All packets encrypted – firewall cannot detect the migration

State Transfer via Redis (observed with MONITOR)

```
1781583670.528 [0 141.217.168.152:44478] "SET" "quic migration:default" "4d494752..." "EX" "60"
1781583670.567 [0 141.217.168.143:58598] "GET" "quic migration:default"
1781583670.568 [0 141.217.168.143:58606] "DEL" "quic_migration:default"
```

- Primary (.152) writes 358 bytes of migration state (TLS secrets + CIDs)
- Preferred (.143) reads it 39ms later
- Preferred deletes it 1ms after reading (security)
- State only exists in Redis for ~40ms

Migration State Contents (358 bytes)

Field	Size	Description
Magic “MIGR”	4 bytes	Format identifier
QUIC Version	4 bytes	Version 2 (RFC 9369)
Write Traffic Secret	32 bytes	Server-to-client encryption key
Write Next Secret	32 bytes	For TLS key rotation
Read Traffic Secret	32 bytes	Client-to-server decryption key
Read Next Secret	32 bytes	For TLS key rotation
Cipher + metadata	~25 bytes	AES_128_GCM, epoch, packet numbers
8 Local CIDs	~90 bytes	Connection IDs client uses to reach us
1 Remote CID	~12 bytes	Connection ID we use to reach client
Client address	7 bytes	141.217.168.127:port

Files

File	Description
quic_firefox.pcap	Wireshark capture of Firefox migration
quic_migration.pcap	Wireshark capture of neqo-client migration
benchmarks/results/*.csv	Raw benchmark data
benchmarks/results/log_*	Individual test logs